

Semantic Analysis

Slides based on material
by Ras Bodik available at
<http://inst.eecs.berkeley.edu/~cs164/fa04>

Symbol Table

Outline

- How to build symbol tables
- How to use them to find
 - multiply-declared and
 - undeclared variables.

Errori di
Dichiarazione

The Compiler So Far

Front-End
Compiler

- **Lexical analysis** Lexer
 - Detects inputs with illegal tokens
 - e.g.: main£ ();
- **Parsing** Parser
 - Detects inputs with ill-formed parse trees
 - e.g.: missing semicolons
- **Semantic analysis**
 - Last “front end” phase
 - Catches all remaining errors

Introduction

L'Analisi Semantica agisce come un "controllore del significato" subito dopo l'Analisi Sintattica (che controlla la grammatica). Mentre la sintassi verifica se le parole sono disposte correttamente, la semantica verifica se le parole hanno senso nel contesto del linguaggio e del programma.

- typical semantic errors:

Tenta di dichiarare la stessa entità (variabile, funzione, classe, ecc.) più di una volta nello stesso ambito (scope).

multiple declarations: a variable should be declared (in the same scope) at most once

Perché è un errore: Genera ambiguità per il compilatore. Se dichiarare `int x;` e poi di nuovo `float x;` nello stesso blocco di codice, il compilatore non sa quale tipo o quale `x` usare in seguito.

Tenta di utilizzare un identificatore che non è stato precedentemente introdotto nell'ambito corrente o in un ambito accessibile.

undeclared variable: a variable should not be used before being declared.

Perché è un errore: Il compilatore non può assegnare una locazione di memoria o un tipo a un identificatore sconosciuto. In termini tecnici, non trova un'associazione per l'identificatore nella tabella dei simboli.

Un'operazione (ad esempio, un'assegnazione o un'operazione aritmetica) viene eseguita su operandi di tipi incompatibili che non possono essere convertiti automaticamente (coercizione).

type mismatch: e.g. type of the left-hand side of an assignment should match the type of the right-hand side.

Una funzione o un metodo viene chiamato con un numero di parametri effettivi diverso dal numero di parametri formali attesi, oppure i tipi dei parametri effettivi non corrispondono ai tipi attesi (o non sono compatibili).

wrong argument number/types: methods should be called with the right number and types of arguments.

Perché è un errore: La funzione è stata progettata per lavorare con un set specifico di dati. Fornire dati diversi o in quantità diversa impedisce l'esecuzione logica della funzione.



La seconda fase si svolge dopo che tutti i nomi sono stati risolti e l'AST è stato arricchito con i puntatori.

- Visita Bottom-Up: Inizia dalle foglie dell'AST e risale verso la radice. Questo è fondamentale per determinare il tipo di espressioni complesse.
- Uso del Puntatore alla Tabella dei Simboli: Per ogni nodo ID nell'AST, il compilatore usa il puntatore allegato durante la Fase 1 per recuperare il tipo di quella variabile.
- Verifica degli Errori di Tipo: In corrispondenza dei nodi operatore (come +, =, o chiamate di funzione), il compilatore:

Determina il Tipo dell'Espressione: Basandosi sui tipi degli operandi (ottenuti tramite l'analisi dei nodi figli), calcola il tipo risultante dell'operazione.

Cerca Disallineamenti: Verifica se l'operazione è valida per quei tipi o se i tipi in un'assegnazione sono compatibili. Se non lo sono, segnala l'errore semantico di "type mismatch".

A simple semantic analyzer

Symbol Table = mantiene le entry delle dichiarazioni dei nomi, che vengono collegate agli usi di tali nomi

Siamo in un linguaggio tipizzato staticamente

works in two phases

- ^{visitando} **by visiting** the AST created by the parser

Obiettivo della Fase 1: Risolvere i nomi ed assicurare che tutte le entità utilizzate siano state dichiarate in modo univoco e accessibile.

1. Generation of Enriched AST (performed top-down)

For each scope in the program:

process the declarations =

- add new entries to the **symbol table** and
- report any variable/method that is **multiply declared**

process the statements =

- find uses of **undeclared variables**, and
- in **"ID" nodes** (variable/method **usages**) of the AST add a pointer to the appropriate **symbol table entry**.

2. Type Checking (performed bottom-up) *

Process all of the statements in the program again,

- use **attached symbol-table entry information** to determine the **type of each expression**, and to find type errors.

Quando la visita raggiunge il codice che usa le variabili:

- Rilevamento di Variabili Non Dichiarate: Se incontra un nodo ID (identificatore, un uso di variabile/metodo) e non riesce a trovarlo in nessuno degli scope visibili nella Tabella dei Simboli, segnala l'errore semantico di "variabile non dichiarata".
- Arricchimento dell'AST (Linking): Se l'identificatore viene trovato, il nodo ID nell'AST viene arricchito aggiungendo un puntatore diretto all'entry corrispondente nella Tabella dei Simboli.

Questa prima fase è focalizzata sulla gestione dello scope e sulla costruzione del contesto (la Tabella dei Simboli).

- Il compilatore visita l'AST dall'alto verso il basso.

- Aggiunta alla Tabella dei Simboli: Quando incontra una dichiarazione, crea una nuova entry nella Tabella dei Simboli. Questa entry contiene nome, tipo e scope.

- Rilevamento di Dichiarazioni Multiple: Se, tentando di aggiungere una nuova entry, scopre che un identificatore con lo stesso nome esiste già nello stesso scope, segnala immediatamente l'errore semantico di "dichiarazione multipla".

Obiettivo della Fase 2: Verificare la coerenza dei tipi e annotare ciascun nodo dell'espressione nell'AST con il suo tipo calcolato. Questo è cruciale per le fasi successive del compilatore.

Symbol Table = map from names to entries

- **purpose:**
 - keep track of names declared in the program
 - names of
 - variables, classes, fields, methods, ETC....
- **symbol table entry:** informazioni sul nome
 - set of attributes associated with a name, per esempio e.g.:
 - kind of name (variable, class, field, method, etc)
 - type (int, float, etc)
 - nesting level
 - memory location (i.e., where it will be found at runtime).

Scoping

→ Il ST può cambiare in base al tipo di scoping usato dal linguaggio di programmazione

- symbol table design influenced by what kind of **scoping** is used by the compiled language
- In most languages, the same name can be **declared multiple times**
 - if its declarations occur in **different scopes**, and/or
 - involve **different kinds of names**.

Scoping: example

- Java: can use same name for
 - a class,
 - field of the class,
 - a method of the class, and
 - a local variable of the method
- *legal Java program:*

```
class Test {  
    int Test;  
    void Test( ) { double Test; }  
}
```

Il nome é costituito da nome metodo
+ tipo dei parametri (signature)

Quando il compilatore incontra una chiamata `sum(a, b)`, non gli basta trovare il nome `sum`. Deve fare un'ulteriore verifica:

- Determina il tipo degli argomenti effettivi (`a` e `b`) (questo richiede in realtà un'anticipazione del Type Checking della Fase 2).
- Cerca nella Tabella dei Simboli l'entry per `sum` che corrisponde esattamente o è compatibile con i tipi degli argomenti forniti.

Scoping: overloading

- **Java and C++** (but not in Pascal or C):
 - can use the same name for more than one method
 - as long as the number and/or types of parameters are unique.

Numero di parametri (es. `sum(int a)` vs `sum(int a, int b)`).
Tipo dei parametri (es. `print(int x)` vs `print(String s)`).

`int add(int a, int b);`

`float add(float a, float b);`

Le regole di scope di un linguaggio sono fondamentali per l'Analisi Semantica perché stabiliscono la visibilità degli identificatori (variabili, funzioni, classi) e risolvono l'ambiguità.

Funzione Principale: Le regole determinano, per ogni utilizzo di un elemento con nome (es. x), quale specifica dichiarazione di quell'elemento si intende. In termini di compilatore, mappano l'uso alla sua dichiarazione nella Tabella dei Simboli.

Scoping: general rules

- **The scope rules of a language:**
 - determine which declaration of a named element (e.g. a variable) corresponds to each use of the element.
 - i.e., scoping rules map uses of element to their declarations.
- **C++ and Java use static scoping:**
 - mapping from uses to declarations is made at compile time.
 - C++ uses the "most closely nested" rule
 - a use of variable x matches the declaration in the most closely enclosing scope such that the declaration precedes the use.
 - inner scope variable x declaration hides x declared in an outer scope.
 - in Java:
 - inner scopes cannot define variables defined in outer scopes

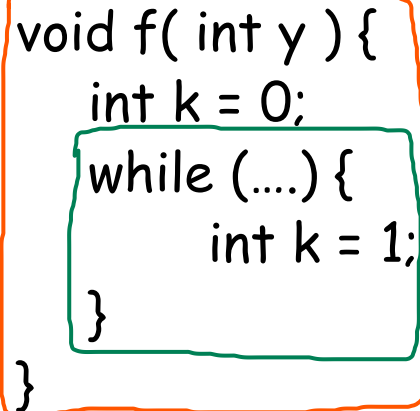
Si differenzia dallo Scoping Dinamico, dove la mappatura dipende dalla sequenza di chiamate di funzione a runtime.

Viene scelta la dichiarazione trovata nello scope più vicino e che precede testualmente il punto d'uso.

Scope levels

- Each function has one or more scopes:
 - one for the parameters and the function body,
 - and possibly additional scopes in the function
 - each nested block (delimited by curly braces)

Example (assume C++ rules)



The diagram shows a C++ function definition: `void f( int y ) { int k = 0; while (....) { int k = 1; } }`. The function body is enclosed in an orange box, and the while loop body is enclosed in a green box. A blue arrow points from the text 'Outmost scope che include in esso la funzione f' to the orange box.

```
void f( int y ) {  
    int k = 0;  
    while (....) {  
        int k = 1;  
    }  
}
```

Outmost
scope che
include in esso
la funzione f

// y is a parameter
// k is a local variable

// another local var, in a loop (not ok in Java)

- ⊖ the outmost scope includes just the name "f", and
- function f itself has two inner (nested) scopes:
 - ① The scope for the body of f, which includes parameter y and local variable k that is initialized to 0.
 - ② The innermost scope is for the body of the while loop, and includes the variable k that is initialized to 1.

TEST YOURSELF

- This is a C++ program. Match each use to its declaration, or say why it is a use of an undeclared variable.

```
class Foo {  
    int k=10, x=20;  
    void foo(int k) {  
        int a = x;  
        int x = k;  
        int b = x;  
        while (...) {  
            int x=11;  
            if (x == k) {  
                int k, y;  
                k = (y = x);  
            }  
            if (x == k) { int x, y; }  
        }  
    }  
}
```

Questo k nella funzione foo, non é lo stesso dichiarato nello scope della classe Foo

Lo scoping dinamico è un modello di risoluzione dei nomi in cui un identificatore (variabile) viene associato alla sua dichiarazione in base alla sequenza di chiamate di funzione attive (lo stack di esecuzione), non alla posizione del codice nel file sorgente.

Dynamic scoping

Storicamente, si usano perché molto semplice da implementare rispetto al Static Scoping

- Not all languages use static scoping.
- Lisp, APL, and Snobol use dynamic scoping.
- Other languages, like Common Lisp or Perl, allow the programmer to choose among static or dynamic scoping
- Dynamic scoping:
 - Use of a variable with no corresponding declaration in the same function (non-local reference)
 - corresponds to the declaration in most-recently-called still active function.



La regola che governa lo Scoping Dinamico è la seguente: Per un uso di variabile non locale (cioè, quando la variabile non è dichiarata nella funzione corrente), la dichiarazione corrispondente è quella presente nella funzione ancora attiva (nello stack) che è stata chiamata più di recente.

Significato per la Tabella dei Simboli (Run-time): Quando un'uso di variabile non è risolto localmente, il compilatore cerca la sua dichiarazione risalendo lo stack di chiamate (stack trace). Si ferma alla prima dichiarazione trovata per quell'identificatore.

Example

- For example, consider the following code:

```
char x;  
void main() { f1(); f2(); }  
void f1() { int x = 10; g(); }  
void f2() { String x = "hello"; f3(); g(); }  
void f3() { double x = 30.5; }  
void g() { print(x); }
```

Con le chiamate, main -> f1 -> g; x = 10

Con le chiamate,
main -> f2 -> g;
x='hello'

↓
Scoping Statico: x si riferisce alla dichiarazione 'char x'

Scoping Dinamico: x dipende dalla sequenza delle chiamate di funzioni che rimangono attive

↙
'hello' perché f3
quando viene
chiamata g, f3 è
già terminata,
quindi non è
presente nello
stack trace delle
chiamate

TEST YOURSELF

Con Scoping Dinamico, guardo lo stack trace delle chiamate delle funzioni ancora attive
Con Scoping Statico, guardo lo scope

- Assuming that static (resp. dynamic) scoping is used, what is output by the following program?

```
{ int x = 0;  
  void f() {int x=1; g() ;}  
  void g() {print x;}  
  f(); }
```

Scoping Statico: Stampa 0
Scoping Dinamico: Stampa 1

Static vs dynamic scoping

- generally, dynamic scoping is a bad idea
 - can make a program difficult to understand
 - a single use of a variable can correspond to
 - many different declarations
 - with different types!

Used before declared?

I campi e metodi di un oggetto di una classe vengono allocati nell'Heap insieme all'oggetto a cui fanno parte, mentre le variabili nello stack

- can a name be used before it is defined?
 - Java: a method or field name can be used before the definition appears,
 - not true for a variable!

Example JAVA

```
class Test {  
    void f() {  
        val = 0;  
        // field val has not yet been declared -- OK  
        g();  
        // method g has not yet been declared -- OK  
        x = 1;  
        // var x has not yet been declared -- ERROR!  
        int x;  
    }  
    void g() {}  
    int val;  
}
```

Simplification

Applichiamo delle semplificazioni per capire meglio l'analisi semantica

- From now on, assume that our language:
 - 1 - uses static scoping
 - 2 - requires that all names be declared before they are used
 - 3 - does not allow multiple declarations of a name in the same scope
 - e.g. no method overloading
 - even for different kinds of names
 - e.g. field and method with the same name not allowed
 - 4 - does allow the same name to be declared in multiple nested scopes (simile a C++)
 - but only once per scope

Symbol Table Implementations

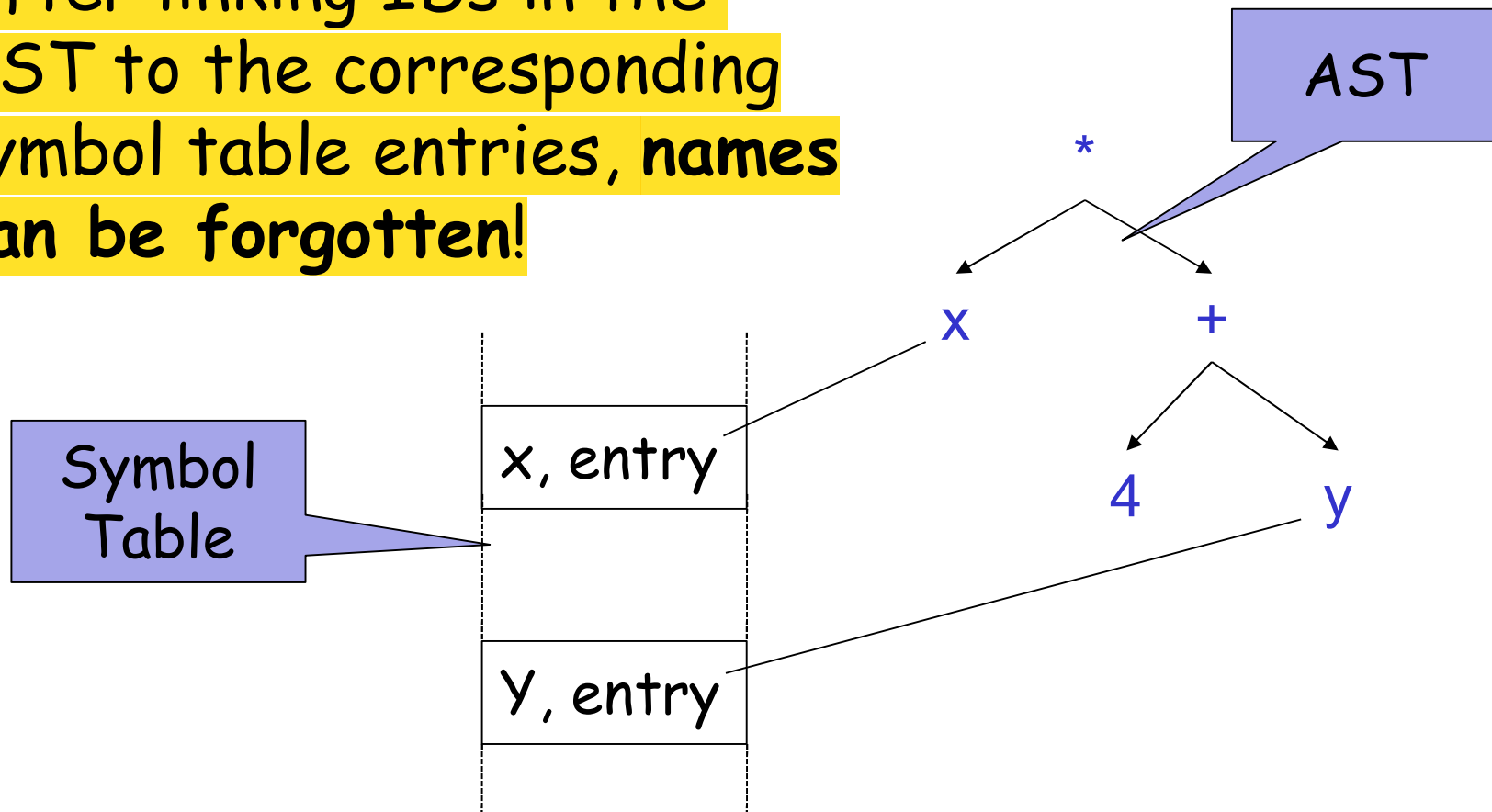
- In addition to the above simplification, assume that the symbol table will be used to answer two questions:
 1. Given a declaration of a name, is there already a declaration of the same name in the current scope
 - i.e., is it multiply declared?
 2. Given a use of a name, to which declaration does it correspond (using the "most closely nested" rule), or is it undeclared?

Note

- The symbol table is only needed to answer those two questions, i.e.
 - 1 - once all declarations have been processed to build the symbol table,
 - 2 - and all uses have been processed to link each ID node in the abstract-syntax tree with the corresponding symbol-table entry,
- then the symbol table itself is no longer needed
 - because no more lookups based on name will be performed

Graphically...

- After linking IDs in the AST to the corresponding symbol table entries, names can be forgotten!



What operation do we need?

- Given the above assumptions, we will need:

1. (for name declarations)

Se la ricerca ha successo, si è verificato un errore di dichiarazione multipla (un errore semantico)

Look up a name in the current scope only

Cerca l'identificatore (name) SOLO nell'ambito di visibilità corrente.

- to check if it is multiply declared

and insert a new name into the symbol table mapped to an entry containing its attributes.

Inserisce il nuovo identificatore nella Tabella dei Simboli, associandolo ai suoi attributi

2. (for name usages)

Look up a name in the current and enclosing scopes

Cerca l'identificatore, partendo dallo scope corrente e risalendo in modo gerarchico tutti gli scope annidati esterni (enclosing scopes). Si ferma alla prima corrispondenza trovata

- to check for a use of an undeclared name, and
- to link a use with the corresponding symbol-table entry.

3. Do what must be done when a new scope is entered.

Crea un nuovo scope nella struttura dati della Tabella dei Simboli.

4. Do what must be done when a scope is exited.

Rimuove l'ambito corrente, rendendo inaccessibili (e liberabili in memoria) tutte le dichiarazioni locali che conteneva. Questo è fondamentale per prevenire l'accesso errato a variabili fuori scope.

Two possible symbol table implementations

1. a list of tables Lista di Tabelle
2. a table of lists Tabella di Liste

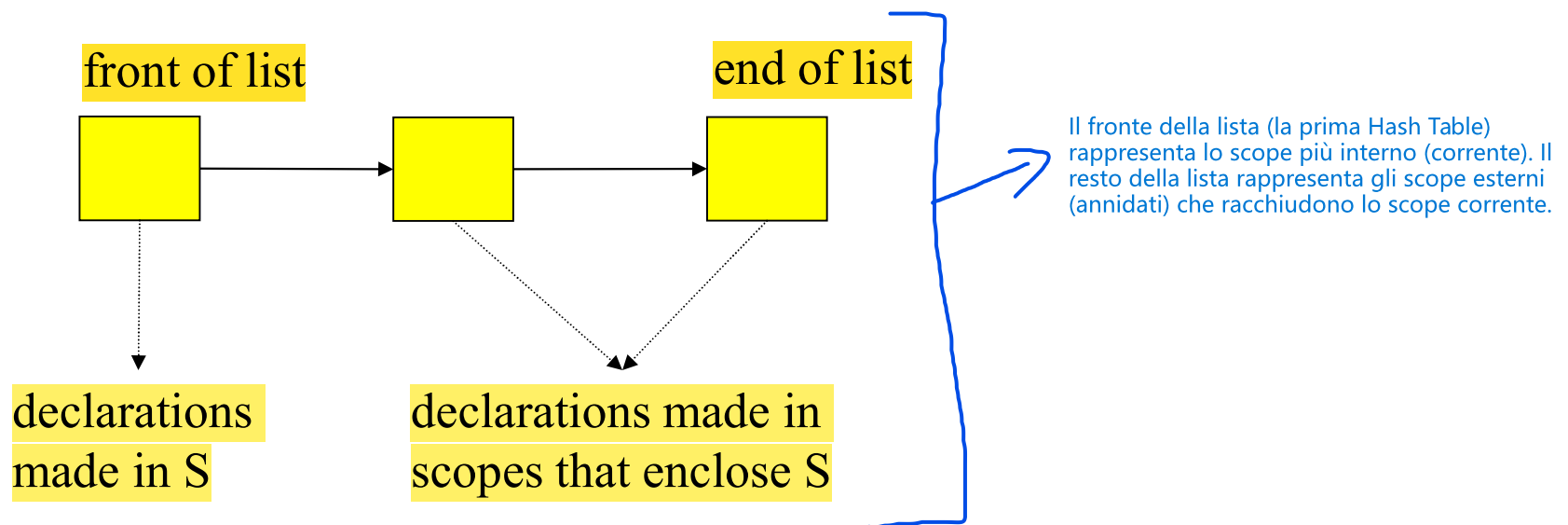
- For each approach, we will consider
 - what must be done when processing a declaration,
 - when processing a use, and
 - when entering and exiting a scope.
- Simplification:
 - assume each symbol-table entry includes only:
 - its type and the nesting level of its declaration

Method 1: List of Hashtables

L'idea centrale di questa implementazione è quella di rappresentare la gerarchia degli scope attivi utilizzando una lista, dove ogni nodo della lista è un'hash table dedicata a un singolo scope.

- The idea:
 - symbol table = a list of hashtables,
 - one hashtable for each currently visible scope.
- When processing a scope S:

→ Ogni singola Hash Table nella lista memorizza tutte le dichiarazioni (variabili, funzioni, classi) che appartengono esclusivamente a un determinato scope attualmente visibile.

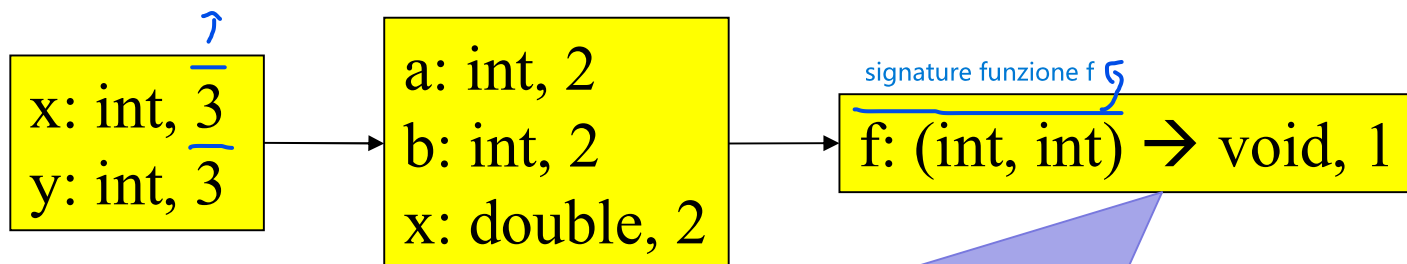


Example:

Alla creazione di una nuova hashtable, viene incrementata la variabile globale che mantiene il conto del nesting level in cui si é attualmente

```
void f(int a, int b) {  
    double x;  
    while (...) { int x, y; ... }  
}  
void g() { f(); }
```

- After processing declarations inside the while loop:



Method type:
function with domain → codomain

List of hashtables: the operations

1. On scope entry:

Entrata nello scope

- increment the current level number and add a new empty hashtable to the front of the list.

2. To process a declaration of x:

Processa una dichiarazione x

- look up x in the first table in the list.
 - If it is there, then issue a "multiply declared variable" error;
 - otherwise, add x to the first table in the list mapped to a new entry containing x type and nesting level.

... continued

3. To process a use of x: Processa un uso di x

- look up x starting in the first table in the list;
 - if it is not there, then look up x in each successive table in the list.
 - link the use of x with the found symbol-table entry
 - If, instead, x is not in any table then issue an "undeclared variable" error.

4. On scope exit, Uscita da scope

- remove the first table from the list and decrement the current level number.

Remember

il nome di un metodo appartiene allo scope esterno (contenitore), non allo scope interno del metodo stesso.

- method names belong to the hashtable for the outer scope (w.r.t. inner method scopes)
 - i.e. not to the same table as the method's variables/parameters
- For instance, in the example above:
 - method name f is in the symbol table for the outermost scope
 - name f is not in the same scope as parameters a and b, and variable x.
 - This is so that when the use of name f in method g is processed, the name is found in an enclosing scope's table.

The running times for each operation:

1. **Scope entry:** Entrare in un nuovo scope richiede di creare una nuova, vuota hash table e di aggiungerla in cima alla lista (o allo stack) dei scope attivi.
 - time to initialize a new, empty hashtable;
 - probably proportional to the size of the hashtable.
2. **Process a declaration:** L'inserimento di un nuovo simbolo avviene solo nella Hash Table dello scope corrente (quella in testa alla lista).
 - using hashing, constant expected time ($O(1)$).
3. **Process a use:** Questa è l'operazione più variabile. L'Analisi Semantica deve risalire la catena degli scope (la lista di Hash Table) fino a trovare la dichiarazione
 - using hashing to do the lookup in each table in the list, the worst-case time is $O(\text{depth of nesting})$, when every table in the list must be examined.
4. **Scope exit:** Uscire da uno scope richiede di rimuovere la Hash Table in testa alla lista.
 - time to remove a table from the list, which should be $O(1)$



Punto Debole: complessità temporale proporzionale alla profondità della lista

TEST YOURSELF

- Assume that the symbol table is implemented using a list of hashtables.
- Draw pictures to show how the symbol table changes as each declaration in the following code is processed.

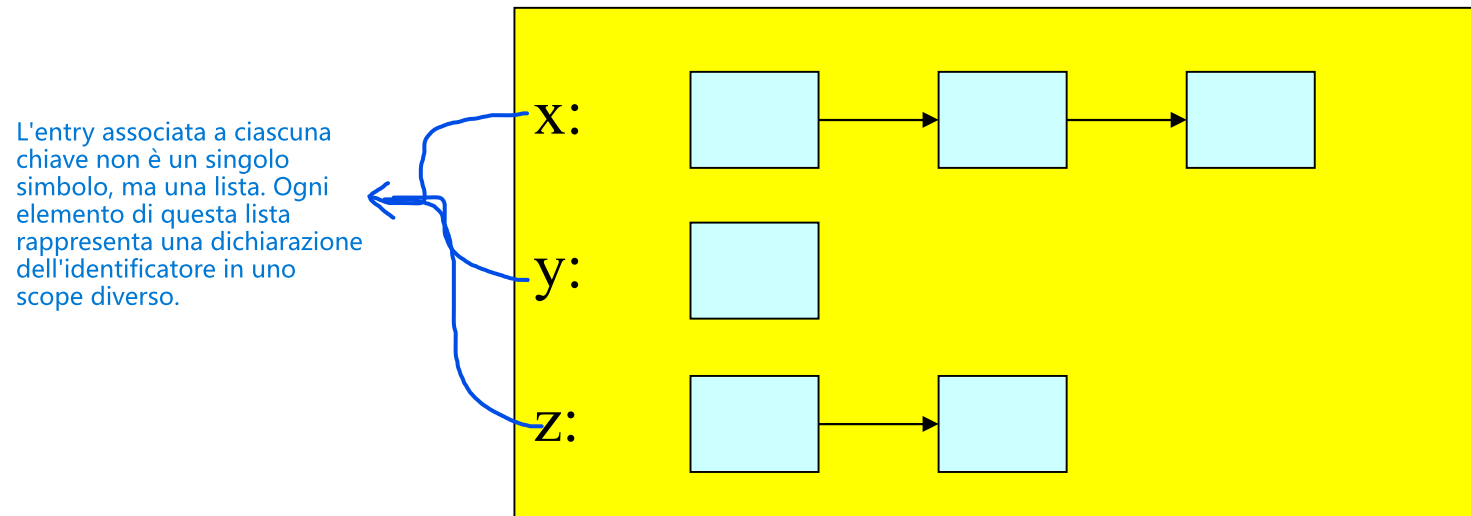
```
void g(int x, int a) {  
    double d;  
    while (...) {  
        int d, w;  
        double x, b;  
        if (...) { int a,b,c; }  
    }  
    while (...) { int x,y,z; }  
}
```

In questo approccio, si utilizza un'unica Hash Table per l'intero programma, e la gestione dello scoping annidato (l'ombreggiamento delle variabili in scope esterni) viene demandata alle liste concatenate associate a ciascun identificatore.

Method 2: Hashtable of Lists

Un'unica tabella di Liste

- the idea:
 - when processing a scope S , the structure of the symbol table is:



Ordine delle Liste: Per implementare la regola del "most closely nested" (scope più interno), le dichiarazioni vengono aggiunte alla testa della lista (come uno stack).

Definition

Esiste una singola Hash Table che funge da struttura di base per l'intera Tabella dei Simboli. Questa tabella contiene una chiave per ogni identificatore (nome di variabile, funzione, ecc.) che è attualmente visibile nello scope corrente o in qualsiasi scope annidato esterno che lo racchiude.

there is just one big hashtable, containing an entry for each name for which there is

- some declaration in scope *S* or
- in a scope that encloses *S*.

Associated with each name is a list of symbol-table entries.

- The first list item corresponds to the most closely enclosing declaration;
- the other list items correspond to declarations in enclosing scopes.

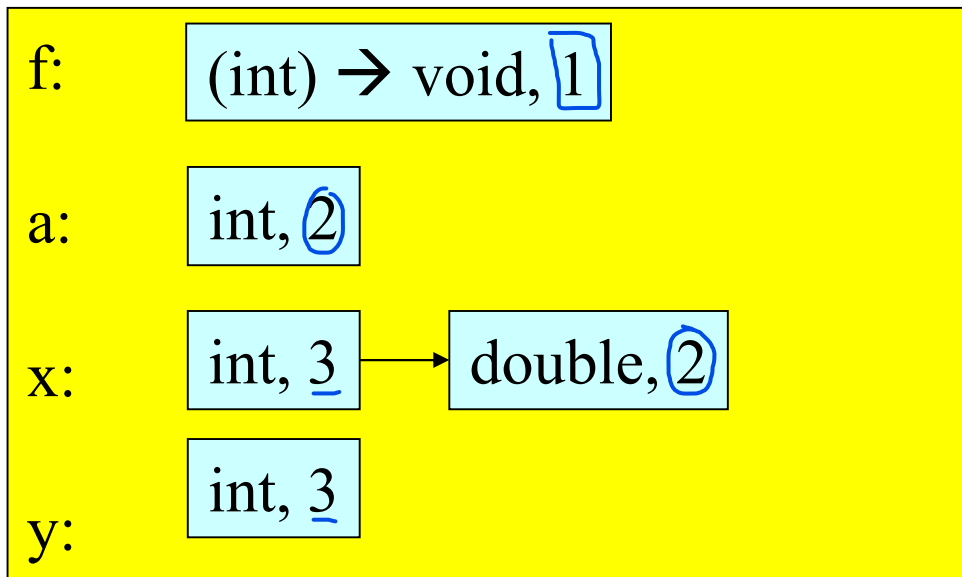
- Lista di Entry: Ad ogni nome nella Hash Table è associata una lista concatenata di Symbol Table Entries (STE). Ogni STE in questa lista rappresenta una dichiarazione di quel nome.
- Ordine Cruciale: L'ordine in cui sono disposte le dichiarazioni nella lista è fondamentale per implementare la regola del "most closely nested" (la più strettamente annidata):
> Testa della Lista: Il primo elemento della lista (quello in testa) corrisponde alla dichiarazione fatta nello scope più interno (più vicino all'uso corrente).
> Elementi Successivi: Gli altri elementi della lista rappresentano le dichiarazioni dello stesso nome che si trovano negli scope via via più esterni che racchiudono lo scope corrente.

Example

All'accesso in uno scope innestato, viene incrementata la variabile globale che mantiene il conto del nesting level in cui si é attualmente

```
void f(int a) {  
    double x;  
    while (...) { int x, y; ... }  
}  
void g() { f(); }
```

- After processing the declarations inside the while loop:



Nesting level information is crucial

- the level-number attribute stored in each list item enables us to determine whether the most closely enclosing declaration was made
 - in the current scope or
 - in an esterno enclosing scope.

Hashtable of lists: the operations

1. On scope entry: Entrata in uno scope
 - increment the current level number. (incrementa la variabile globale che tiene conto del livello di scope in cui siamo)
2. To process a declaration of x: Processa la dichiarazione di x
 - look up x in the symbol table.
 - If x is there, fetch the level number from the first list item.
 - If that level number = current level then issue a "multiply declared variable" error;
 - otherwise, add a new item to the front of the list with the appropriate type and the current level number.

... **continue**

3. **To process a use of x:** Processa l'uso di x

- **Look up x in the symbol table.**
 - **If it is there, link the use of x with the symbol-table entry at the front of the list**
 - **If it is not there, then issue an "undeclared variable" error.**

4. **On scope exit:** Uscita dallo scope

- **Scan all the names in the symbol table, looking at the first item on each list.**
- **If that item's level number = current level number, then remove it from its list**
 - **and if the list becomes empty, remove the name.**
- **Finally, decrement the current level number.**

Running times

1. **Scope entry:** Entrare in un nuovo scope richiede solo di incrementare il contatore del livello di annidamento (nesting level).
 - time to increment the level number, $O(1)$.
2. **Process a declaration:** L'inserimento di un nuovo nome richiede di Trovare l'identificatore nella Hash Table, Controllare la testa della lista per verificare l'ombreggiamento/duplicazione (dipendente dallo scope corrente) ed Aggiungere la nuova entry in testa alla lista
 - using hashing, constant expected time ($O(1)$).
3. **Process a use:** Si trova il nome nella Hash Table. La dichiarazione corretta e più strettamente annidata è garantita essere in testa alla lista: non è necessario scorrere gli scope.
 - using hashing, constant expected time ($O(1)$).
4. **Scope exit:**
 - time proportional to the number of names in the symbol table.

Punto Debole

Uscire da uno scope è l'operazione più costosa. L'analizzatore deve Scorrere l'intera Hash Table (tutti gli N nomi) e Per ogni nome, verificare se l'entry in testa alla sua lista appartiene al livello di scope che si sta chiudendo. Se l'entry appartiene a quello scope, l'entry viene rimossa (POP) dalla testa della lista.

TEST YOURSELF

- Assume that the symbol table is implemented using a hashtable of lists.
- Draw pictures to show how the symbol table changes as each declaration in the following code is processed.

```
void g(int x, int a) {  
    double d;  
    while (...) {  
        int d, w;  
        double x, b;  
        if (...) { int a,b,c; }  
    }  
    while (...) { int x,y,z; }  
}
```

Type Checking

Types

Il concetto di Tipo è una delle astrazioni più importanti in informatica e nella teoria dei linguaggi di programmazione. È la base su cui l'analizzatore semantico verifica la coerenza logica e l'uso corretto dei dati.

Nei linguaggi a tipizzazione forte, la nozione di tipo è rigida e la compatibilità è controllata rigorosamente in fase di compilazione.

Nei linguaggi a tipizzazione debole, le conversioni implicite (coercizioni) sono più comuni, rendendo il concetto di tipo più flessibile e talvolta ambiguo.

- What is a type?
 - The notion varies from language to language

Consensus

Nonostante le differenze specifiche, c'è un forte consenso teorico sulla natura di un tipo in un linguaggio formale:

- A set of values
- A set of operations on those values

- Classes are one instantiation of the modern notion of type

I tipi sono un insieme di valori e per ogni tipo viene identificato un insieme di azioni accettate su quei valori

Why Do We Need Type Systems?

Il linguaggio definisce i tipi e le azioni legate ai tipi: dà una interpretazione ai valori

Consider the assembly language fragment

I sistemi di tipi permettono al compilatore (e quindi all'Analisi Semantica) di fare una verifica statica (compile-time check) sul comportamento previsto del programma.

Identificazione di Errori Logici: Molti errori logici comuni, vengono catturati e segnalati come errori semantici ben prima che il codice venga eseguito.

Garanzia di Senso: La regola fondamentale è: se l'operazione sui tipi è valida, l'operazione ha senso nel contesto del linguaggio.

add \$r1, \$r2, \$r3

In Assembly, non esistono i tipi



What are the types of \$r1, \$r2, \$r3?

I tipi sono una forma di documentazione per il codice.

Interfacce Chiare: In un sistema complesso con molte funzioni e moduli, i tipi definiscono in modo esplicito il contratto tra i moduli: quali input sono attesi e quali output sono prodotti.

Manutenzione Semplificata: Se un programmatore modifica una funzione, il sistema di tipi segnala immediatamente tutti gli altri punti del codice che utilizzano quella funzione e che devono essere adattati, riducendo il rischio di errori a cascata.

In sintesi, la disciplina di tipo trasforma un insieme caotico di istruzioni macchina in un sistema organizzato, scalabile e manutenibile, dove l'Analisi Semantica funge da garante di questo ordine.

Types and Operations

- Most operations are legal only for values of some types
 - It doesn't make sense to add a function pointer and an integer in C
 - It does make sense to add two integers
 - But both have the same assembly language implementation!

Type Systems

Specifica la Validità delle Operazioni: Definisce formalmente un insieme di regole di tipo che specificano, per ogni operatore o costrutto del linguaggio, quali tipi sono operandi legali e quale tipo produce il risultato.

- A language's type system specifies which operations are valid for which types

- The goal of type checking is to ensure that operations are used with the correct types

- Enforces intended interpretation of values, because nothing else will!

- Assicurare l'Uso Corretto: L'obiettivo primario è verificare che le operazioni siano usate con i tipi corretti, come stabilito dalle regole del sistema.
- Enforce Intended Interpretation: Il Type Checking è l'unico meccanismo che garantisce l'interpretazione intenzionale dei valori. Il tipo vincola l'hardware a comportarsi come richiesto dall'astrazione del linguaggio.

- Type systems provide a concise formalization of the semantic checking rules

Concisa Formalizzazione: I sistemi di tipi forniscono un modo conciso e matematicamente rigoroso per esprimere le regole di controllo semantico. Regole di Inferenza: Queste regole sono spesso espresse come Regole di Inferenza di Tipo (Type Inference Rules), che stabiliscono le precondizioni sui tipi degli operandi e il tipo che deve risultare dall'espressione.

What Can Types do For Us?

- Can detect certain kinds of errors
- **Memory errors:** i tipi aiutano a prevenire gli usi errati dei puntatori e degli indirizzi.
 - Reading from an invalid pointer, etc.
- **Violation of abstraction boundaries:**

Questo è un concetto fondamentale nell'ingegneria del software, specialmente in linguaggi orientati agli oggetti. I tipi definiscono i contratti e le interfacce, e l'Analisi Semantica si assicura che questi contratti non vengano violati.

```
class FileSystem {  
  open(x : String) : File {  
    ...  
  }  
  ...  
}
```

```
class Client {  
  f(fs : FileSystem) {  
    File fdesc ← fs.open("foo")  
    ...  
  } -- f cannot see inside fdesc !  
}
```

Il sistema di tipi impedisce alla funzione f di accedere o manipolare i dettagli interni (i campi privati) dell'oggetto fdesc che non sono esposti dall'interfaccia pubblica del tipo File.

Type Checking Overview

- Three kinds of languages:

1 - *Statically typed*: All or almost all checking of types is done as part of compilation (C) Compile-Time

la maggior parte

In Java, parte del type checking viene fatto dalla VM

2 - *Dynamically typed*: Almost all checking of types is done as part of program execution (Scheme) Run-Time

3 - *Untyped*: No type checking (machine code)

The Type Wars

- Competing views on static vs. dynamic typing

- Static typing proponents say:

- Static checking catches many programming errors at compile time
- Avoids overhead of runtime type checks

Cattura degli Errori in Fase di Compilazione: L'Analisi Semantica (Type Checking) rileva la maggior parte degli errori di tipo prima che il codice venga eseguito. Ciò porta a software più robusto e affidabile in produzione.

Evitare l'Overhead a Runtime: Una volta che i tipi sono verificati staticamente, il programma in esecuzione non ha bisogno di includere codice per controllare continuamente la validità dei tipi, risultando in un codice binario più veloce e un'esecuzione più efficiente.

- Dynamic typing proponents say:

- Static type systems are restrictive
 - e.g. due to the type statically associated/fixed for a variable
 - for variable "double x" we cannot do "int y=x" even if at run-time "x" actually contains an integer
 - programmers end up using casts escaping the type system
- Rapid prototyping easier in a dynamic type system

Flessibilità e Meno Restrizioni: I sistemi statici sono percepiti come troppo restrittivi. La flessibilità del Dynamic Typing permette di scrivere codice più generale e conciso. Prototipazione Rapida: La mancanza di rigorosi controlli di tipo in fase di compilazione accelera il ciclo di sviluppo, facilitando l'iterazione e la prototipazione rapida. Si può cambiare la struttura del codice molto più facilmente senza dover rifattorizzare le definizioni di tipo ovunque.

Il Type Checking è il processo con cui l'analizzatore semantico garantisce che il codice rispetti le regole stabilite dal sistema di tipi del linguaggio. Il Type Checking tipicamente visita l'AST (spesso in modalità bottom-up) e verifica, nodo per nodo, che i tipi degli operandi siano compatibili con l'operatore.

Relazione con Type Checking: Molti vedono l'inferenza come parte integrante del Type Checking, specialmente quando si tratta di calcolare il tipo di espressioni complesse. Per verificare la validità di un'espressione, si deve prima inferire il tipo di tutte le sue sotto-espressioni.

In sintesi: Type Checking è la verifica (è il codice corretto?), mentre Type Inference è la deduzione (qual è il tipo di questo codice?). In pratica, si fondono in un unico processo che annota l'AST con le informazioni di tipo risolte.

Type Checking and Type Inference

Type Checking is the process of checking that the program ^{obbedisce} obeys the type system

Often involves inferring types for parts of the program

- Some people call the process *type inference*

L'Inferenza di Tipo è l'aspetto del Type Checking che si occupa di dedurre (inferire) il tipo di un'espressione o di una variabile quando questo non è stato dichiarato esplicitamente dal programmatore. Nei linguaggi moderni, non è sempre necessario dichiarare il tipo di una variabile o di una funzione.

Rules of Inference

- We have seen two examples of formal notation specifying parts of a compiler
 - Regular expressions (for the lexer)
 - Context-free grammars (for the parser)
- The appropriate formalism for type checking is logical rules of inference

Le Regole di Inferenza forniscono un metodo matematicamente rigoroso e conciso per specificare quando un'espressione ha un certo tipo, basandosi sui tipi delle sue sotto-espressioni.

Why Rules of Inference?

- Inference rules have the form
If Hypothesis is true, then Conclusion is true
- Type checking computes via reasoning
If E_1 and E_2 have certain types, then E_3 has a certain type

From English to an Inference Rule

- The notation is easy to read (with practice)
- Start with a simplified system and gradually add features
- Building blocks
 - Symbol $\wedge$ is “and”
 - Symbol $\Rightarrow$ is “if-then”
 - $x:T$ is “x has type T”

From English to an Inference Rule (2)

If e_1 has type Int and e_2 has type Int ,
then $e_1 + e_2$ has type Int

$(e_1 \text{ has type } \text{Int} \wedge e_2 \text{ has type } \text{Int}) \Rightarrow$
 $e_1 + e_2 \text{ has type } \text{Int}$

$(e_1 : \text{Int} \wedge e_2 : \text{Int}) \Rightarrow e_1 + e_2 : \text{Int}$

From English to an Inference Rule (3)

The statement

$$(e_1: \text{Int} \wedge e_2: \text{Int}) \Rightarrow e_1 + e_2: \text{Int}$$

is a special case of

$$(\text{Hypothesis}_1 \wedge \dots \wedge \text{Hypothesis}_n) \Rightarrow \text{Conclusion}$$

The latter is an inference rule usually written:

$$\frac{\vdash \text{Hypothesis}_1 \quad \dots \quad \vdash \text{Hypothesis}_n}{\vdash \text{Conclusion}}$$

...and the above rule is written:

$$\frac{\vdash e_1 : \text{Int} \quad \vdash e_2 : \text{Int}}{\vdash e_1 + e_2 : \text{Int}}$$

Notation for Inference Rules

- By tradition inference rules are written

$$\frac{\vdash \text{Hypothesis}_1 \quad \dots \quad \vdash \text{Hypothesis}_n}{\vdash \text{Conclusion}}$$

- Type rules have hypotheses and conclusions of the form:

$$\vdash e : T$$

- $\vdash$ means “we can prove that . . .”

An example

- An inference system for proving $x < y$

$\frac{}{\vdash x < (x+1)}$	[A] (Axiom stating that every number is strictly smaller than its successor)
$\frac{\vdash x < y \quad \vdash y < z}{\vdash x < z}$	[T] (Transitivity of $<$)

An example (continued)

- How to prove that $6 < 10$?

$$\vdash 6 < 10$$

An example (continued)

- How to prove that $6 < 10$?

$$\vdash 6 < 7$$

$$\vdash 7 < 10$$

[T]

$$\vdash 6 < 10$$

An example (continued)

- How to prove that $6 < 10$?

$$\frac{\frac{}{\vdash 6 < 7} [A] \quad \vdash 7 < 10}{\vdash 6 < 10} [T]$$

An example (continued)

- How to prove that $6 < 10$?

$$\begin{array}{c} \frac{}{\vdash 6 < 7} [A] \quad \frac{\vdash 7 < 9 \quad \vdash 9 < 10}{\vdash 7 < 10} [T] \\ \hline \vdash 6 < 10 \end{array}$$

An example (continued)

- How to prove that $6 < 10$?

$\frac{}{\vdash 6 < 7}$	$\frac{}{\vdash 7 < 9}$	$\frac{}{\vdash 9 < 10}$	[A]
[A]			[T]
$\vdash 7 < 10$			
$\vdash 6 < 10$			[T]

An example (continued)

- How to prove that $6 < 10$?

$$\begin{array}{c} \begin{array}{c} \begin{array}{c} \text{[T]} \quad \frac{\frac{\frac{}{\vdash 6 < 7} \text{[A]} \quad \vdash 7 < 8 \quad \vdash 8 < 9}{\vdash 7 < 9} \text{[A]} \quad \vdash 9 < 10 \text{[T]}}{\vdash 7 < 10} \text{[T]} \end{array} \\ \hline \vdash 6 < 10 \end{array} \end{array}$$

An example (continued)

- How to prove that $6 < 10$?

$$\begin{array}{c}
 \begin{array}{c}
 [A] \frac{}{\vdash 7 < 8} \quad \frac{}{\vdash 8 < 9} [A] \\
 [T] \frac{}{\vdash 7 < 9} \quad \frac{}{\vdash 9 < 10} [A] \\
 [A] \frac{}{\vdash 6 < 7} \quad \frac{}{\vdash 7 < 10} [T] \\
 \hline
 \vdash 6 < 10
 \end{array}
 \end{array}$$

Type system: Two Rules

$$\frac{(i \text{ is an integer token})}{\vdash i : \text{Int}} \quad [\text{Int}]$$

(this kind of premises are usually written as side conditions, as they do not require other inferences)

$$\frac{}{\vdash i : \text{Int}} \quad [\text{Int}] \quad (i \text{ is an integer token})$$

$$\frac{\begin{array}{c} \vdash e_1 : \text{Int} \\ \vdash e_2 : \text{Int} \end{array}}{\vdash e_1 + e_2 : \text{Int}} \quad [\text{Add}]$$

Type system: Two Rules (Cont.)

- These rules give templates describing how to type integers and + expressions
- By filling in the templates, we can produce complete typings for expressions

$$\frac{\frac{}{\vdash 1 : \text{Int}} \quad \frac{}{\vdash 2 : \text{Int}}}{\vdash 1 + 2 : \text{Int}}$$

Soundness

- A type system is sound if
 - Whenever we can infer $\vdash e : T$
 - Then at run-time e actually evaluates to a value of type T
- We only want sound rules

Type Checking Proofs

- Type checking proves facts $e : T$
 - One type rule is used for each kind of expression, e.g. previous rule used if e is in the form $e_1 + e_2$
- In the type rule used for typing e :
 - The hypotheses are the proofs of types of e 's subexpressions
 - The conclusion is the proof of type of e

Rules for Constants

$$\frac{}{\vdash \text{false} : \text{Bool}} \quad [\text{Bool}]$$
$$\frac{}{\vdash s : \text{String}} \quad [\text{String}] \quad (s \text{ is a string constant})$$

Object Creation Example

$$\frac{}{\vdash \text{new } C(): C} \quad [\text{New}] \quad (C \text{ denotes a class with parameterless constructor})$$

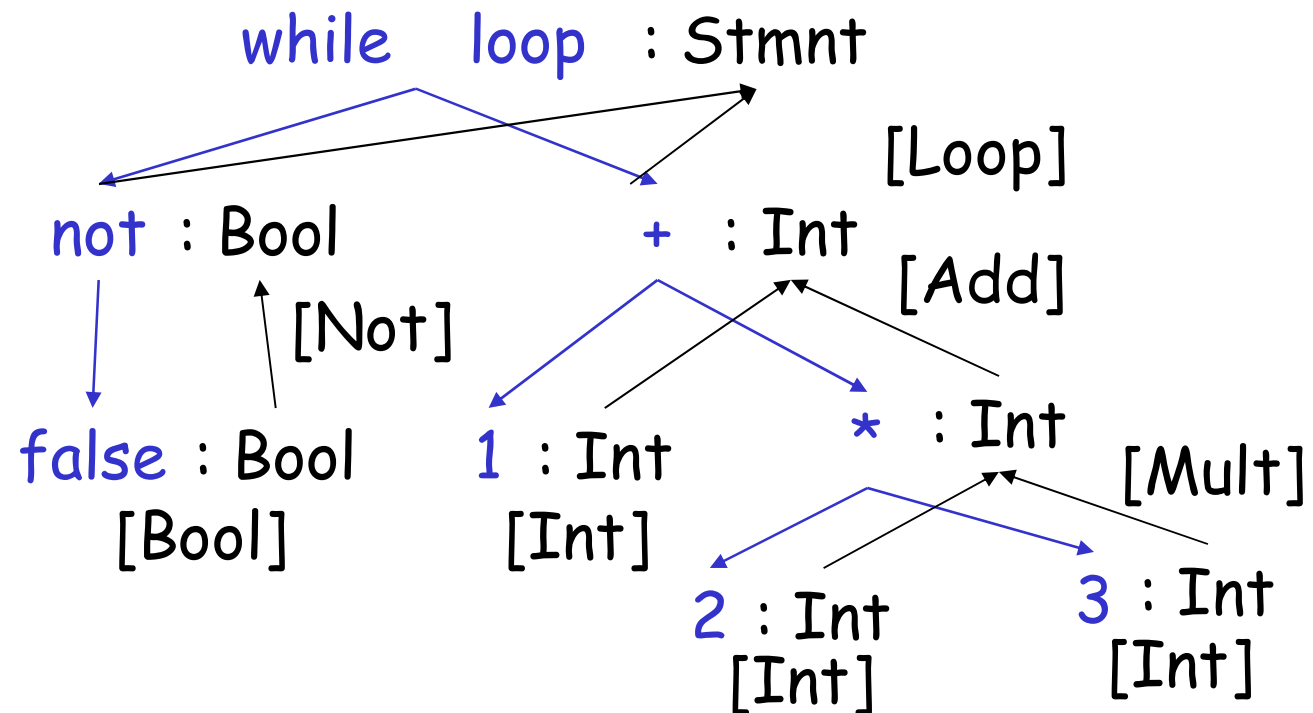
Two More Rules

$$\frac{\vdash e : \text{Bool}}{\vdash \text{not } e : \text{Bool}} \quad [\text{Not}]$$

$$\frac{\begin{array}{c} \vdash e_1 : \text{Bool} \\ \vdash e_2 : T \end{array}}{\vdash \text{while } e_1 \text{ loop } e_2 \text{ pool} : \text{Stmnt}} \quad [\text{Loop}]$$

Typing: Example

- Typing for `while not false loop 1 + 2 * 3 pool`



Typing Derivations

- The typing reasoning can be expressed as a tree:

$$\frac{\frac{\frac{}{\vdash \text{false} : \text{Bool}}}{\vdash \text{not false} : \text{Bool}} \quad \frac{\frac{}{\vdash 1 : \text{Int}} \quad \frac{\frac{\frac{}{\vdash 2 : \text{Int}} \quad \frac{}{\vdash 3 : \text{Int}}}{\vdash 2 * 3 : \text{Int}}}{\vdash 1 + 2 * 3 : \text{Int}}}{\vdash \text{while not false loop } 1 + 2 * 3 \text{ pool} : \text{Stmtnt}}}$$

- The root of the tree is the whole expression
- Each node is an instance of a typing rule
- Leaves are the rules with no hypotheses

A Problem

- What is the type of a variable usage?

$$\frac{}{\vdash x : ?} \quad [\text{Id}] \quad (\text{x is an identifier})$$

- This rule does not have enough information to give a type.
 - We need an **assumption** of the form “we are in the scope of a declaration of x with type T ”)

A Solution: Put more information in the rules!

Let O be a function from **Identifiers** to **Types**

The sentence $O \vdash e : T$

is read:

Under the **assumption** that the **variables** in the **current scope** have the **types** given by O , it is provable that expression e has type T

- **NOTE:** Corresponds to the information stored in the **symbol table**!

Type Environments

- A type environment $\mathcal{O}$ gives **types** for *free* variables
 - A variable is *free* in an (sub)expression if:
 - The expression contains an occurrence of the variable that refers to a declaration outside the expression
 - E.g. in “int f(int x) {return x+y}” only “y” is free
 - E.g. in the (sub)expression “x”, the variable “x” is free

Modified Rules

The type environment is added to the earlier rules:

$$\frac{}{O \vdash i : \text{Int}} \quad [\text{Int}] \text{ (i is an integer)}$$

$$\frac{\begin{array}{c} O \vdash e_1 : \text{Int} \\ O \vdash e_2 : \text{Int} \end{array}}{O \vdash e_1 + e_2 : \text{Int}} \quad [\text{Add}]$$

New Rules

And we can write new rules:

$$\frac{}{O \vdash x : T} \quad [\text{Id}] \quad (\text{if } O(x) = T)$$

Function invocation

The type of a function is usually written:

- $T_1, \dots, T_n \rightarrow T$
- with $T_1, \dots, T_n$ types of the input parameters
- and T the output return type

$$\frac{O \vdash f:(T_1, \dots, T_n \rightarrow T) \quad O \vdash e_1:T_1 \quad \dots \quad O \vdash e_n:T_n}{O \vdash f(e_1, \dots, e_n) : T} \quad [\text{FunCall}]$$

Let (from languages like ML)

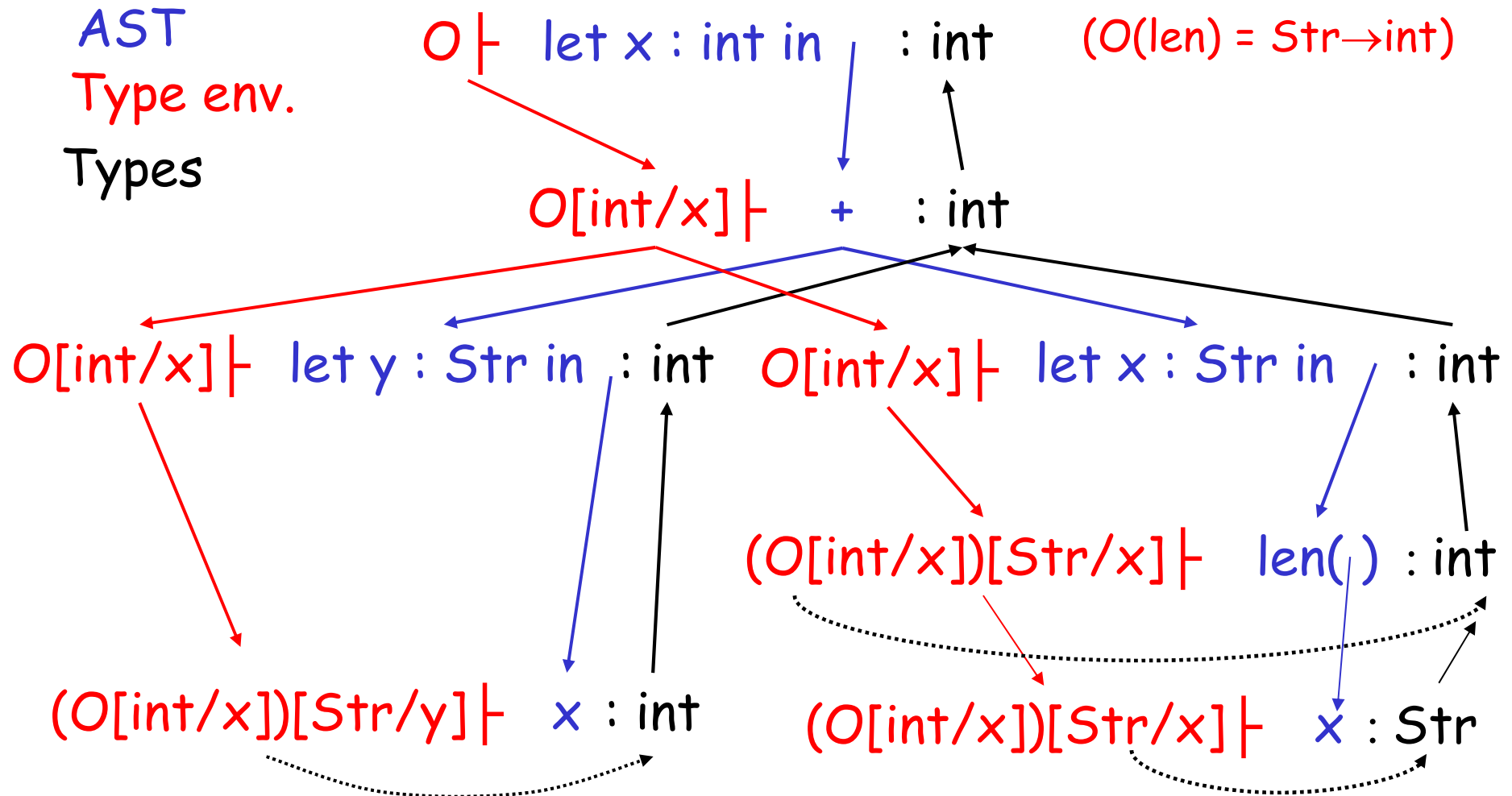
- Let statement declares a variable x with given type T_0 that is then defined throughout e_1 :
 - $\text{let } x : T_0 \text{ in } e_1$ (without initialization)
 - $\text{let } x : T_0 \leftarrow e_0 \text{ in } e_1$ (with initialization)

$$\frac{O[T_0/x] \vdash e_1 : T_1}{O \vdash \text{let } x : T_0 \text{ in } e_1 : T_1} \quad [\text{Let-No-Init}]$$

Let. Example.

- Consider the expression
$$\text{let } x : T_0 \text{ in } ((\text{let } y : T_1 \text{ in } E_{x,y}) + (\text{let } x : T_2 \text{ in } F_{x,y}))$$
(where $E_{x,y}$ and $F_{x,y}$ are some expressions that may contain occurrences of “x” and “y”)
- Declaration
 - of “y” applies to $E_{x,y}$
 - of outer “x” applies to $E_{x,y}$
 - of inner “x” applies to $F_{x,y}$
- This is captured precisely in the typing rule.

Let Example.



Notes

- The type environment gives types to the free identifiers in the current scope
 - The type environment is passed down the AST from the root towards the leaves
 - Types are computed up the AST from the leaves towards the root
- NOTE: in compiler implementations, the “downward” phase generates an **enriched version of the AST** where the **identifiers** are linked to their **symbol table entry** (indicating their type)

Let with Initialization

Consider a **let** with initialization:

$$\frac{\begin{array}{c} O \vdash e_0 : T_0 \\ O[T_0/x] \vdash e_1 : T_1 \end{array}}{O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1} \text{ [Let-Init]}$$

This rule is too “restrictive”. Why?

Let with Initialization

- Consider the example:

class C inherits P { ... }

...

let $x : P \leftarrow \text{new } C$ in ...

...

- The previous let rule does not allow this code
 - We say that the rule is too “restrictive”

Subtyping

- Define a relation $X \leq Y$ on classes to say that:
 - An object of type X can be used when one of type Y is acceptable, or equivalently
 - X conforms with Y
- Also known as “ X is a subclass of Y ”
 - Consider a relation $\leq$ on classes
$$X \leq X$$
$$X \leq Y \text{ if } X \text{ inherits from } Y$$
$$X \leq Z \text{ if } X \leq Y \text{ and } Y \leq Z$$

Let with Initialization (Again)

$$\frac{\begin{array}{c} O \vdash e_0 : T \\ T \leq T_0 \\ O[T_0/x] \vdash e_1 : T_1 \end{array}}{O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1} [\text{Let-Init}]$$

- Both rules for let are sound
- But more programs type check with the latter, which is more “flexible”

Let with Subtyping. Notes.

- There is a tension between
 - “Flexible” rules that do not constrain programming
 - “Restrictive” rules that ensure safety of execution

Expressiveness of Static Type Systems

- A **static type system** enables a compiler to detect many common programming errors
- The cost is that **some correct programs are disallowed**
 - Some argue for dynamic type checking instead
 - Others argue for more expressive static type checking
- But more expressive type systems are also **more complex**

Static and Dynamic Types

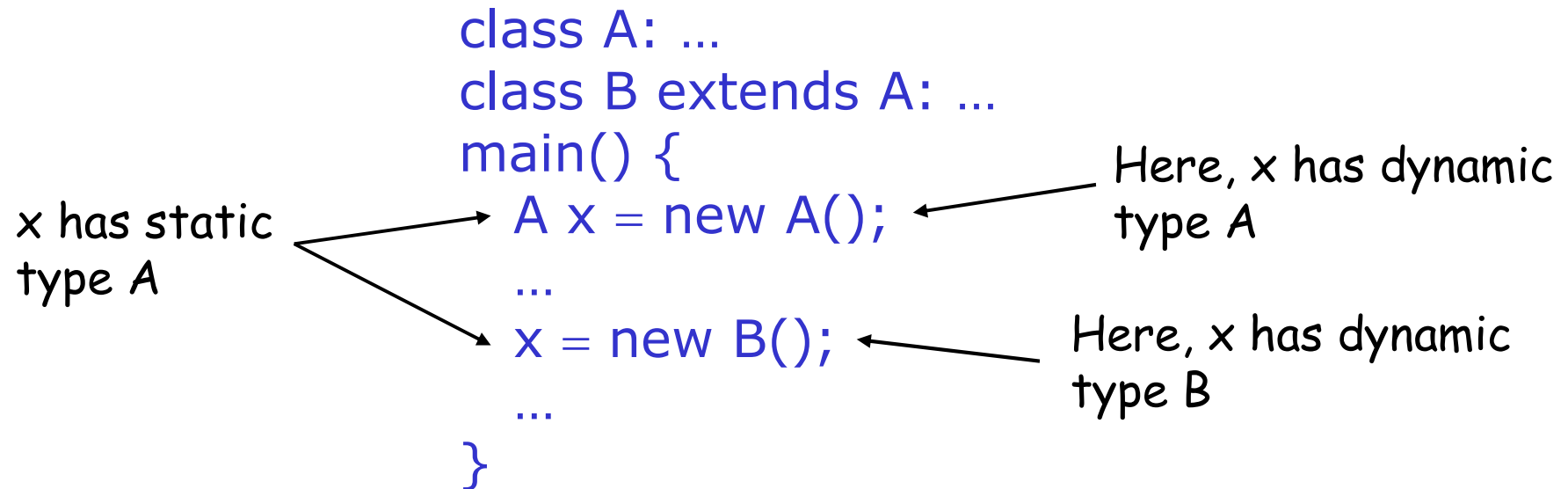
- The *static type* of an object variable is the class *C* used to **declare** the variable.
 - This notion is then extended to **expressions** over object variables.
 - A **compile-time** notion
- The *dynamic type* of an object variable is the class *C* that is used to create its object value (“*new C*”).
 - This notion is then extended to **expressions** over object variables.
 - A **run-time** notion

Static and Dynamic Types. (Cont.)

- In early type systems static types correspond directly with dynamic types
- Soundness theorem: for all expressions E
$$\text{dynamic_type}(E) = \text{static_type}(E)$$

(in all executions, E evaluates to values of the type inferred by the compiler)
- This gets more complicated in **advanced** type systems

Ex. of Code that Type Checks with the Rules



- According to rules: variable x with static type A can be assigned values of type B if $B \leq A$
 - **Liskov** substitution principle:
subtypes can be used in place of supertypes
- Hence: $\text{dynamic_type}(x) \leq \text{static_type}(x)$

Soundness of the Type Checking System

Soundness theorem:

$$\forall E. \text{dynamic_type}(E) \leq \text{static_type}(E)$$

Why is this Ok?

- For E , compiler uses $\text{static_type}(E)$ (call it C)
- All operations that can be used on an object of type C can also be used on an object of type $C' \leq C$
 - Objects of type C' must expose all the public methods and public fields exposed by C
 - Objects of type C' could have additional public methods and fields, but they will be not used if the static type is C

Let. Examples.

- Consider the following class definitions

Class A { a() : Int { return 0 }; }

Class B inherits A { b() : Int { return 1 }; }

- An instance of B has methods “a” and “b”
- An instance of A has method “a”
 - A run-time error occurs if we try to invoke method “b” on an instance of A
- Supertype cannot be used in place of a subtype!

Example of Wrong Let Rule (1)

- Now consider a hypothetical let rule:

$$\frac{O \vdash e_0 : T \quad T \leq T_0 \quad O \vdash e_1 : T_1}{O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1}$$

- How is it different from the correct rule?
- The following good program does not typecheck

`let x : Int ← 0 in x + 1`

- And some bad programs do typecheck

`foo(x : B) : Int { let x : A ← new A() in x.b() }`

Example of Wrong Let Rule (2)

- Now consider another hypothetical let rule:

$$\frac{O \vdash e_0 : T \quad T_0 \leq T \quad O[T_0/x] \vdash e_1 : T_1}{O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1}$$

- How is it different from the correct rule?
- The following bad program is well typed
 $\text{let } x : B \leftarrow \text{new } A() \text{ in } x.b()$
- Why is this program bad?

Recall the correct Let-Init rule

$$\frac{\begin{array}{c} O \vdash e_0 : T \\ T \leq T_0 \\ O[T_0/x] \vdash e_1 : T_1 \end{array}}{O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1} \quad [\text{Let-Init}]$$

- We also need a rule for just assigning a value to an already declared variable x

Assignment of an already declared x

$$\frac{\begin{array}{c} O \vdash e_0 : T \\ T \leq T_0 \\ O \vdash e_1 : T_1 \end{array}}{O \vdash x \leftarrow e_0 ; e_1 : T_1} \quad [\text{Assign}] \quad (\text{if } O(x) = T_0)$$

- E.g. the following program is well typed:
let $x : A \leftarrow \text{new } B() \text{ in } \{ \dots x \leftarrow \text{new } A(); x.a() \}$

Example of Wrong Let Rule (3)

- Now consider another hypothetical let rule:

$$\frac{O \vdash e_0 : T \quad T \leq T_0 \quad O[T/x] \vdash e_1 : T_1}{O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1}$$

- How is it different from the correct rule?
- The following good program is not well typed
 $\text{let } x : A \leftarrow \text{new } B() \text{ in } \{ \dots x \leftarrow \text{new } A(); x.a() \}$
- Why is this program not well typed?

Function invocation with subtyping

Function f with type:

- $T_{0,1}, \dots, T_{0,n} \rightarrow T$
- with $T_{0,1}, \dots, T_{0,n}$ types of the input parameters
- and T the output return type

$$\frac{O \vdash f:(T_{0,1}, \dots, T_{0,n} \rightarrow T) \quad O \vdash e_1:T_1 \dots O \vdash e_n:T_n \quad \forall i \ T_i \leq T_{0,i}}{O \vdash f(e_1, \dots, e_n) : T} \text{ [Fun]}$$

A (generally) wrong subtyping rule

- Let `array of A` be the type of an array containing data of type `A`
- Intuitively, one could assume:
 - If $B \leq A$ then `array of B` $\leq$ `array of A`
(known as "covariant" arrays, present e.g. in Java)
- But, consider the following program (well typed according to this assumption):

```
let function f(x:array of A) {x[1]←new A}  
in let z:array of B  
    in {f(z); z[1].b();};
```
- What is wrong with this program?
 - see next slide...

A (generally) wrong subtyping rule (cont.)

- Problem:
 - When the array of subtypes is used in place of an array of supertypes...
 - ...it is possible to **insert** a supertype in the array...
 - ...and then the supertype can be used in place of a subtype (type error!)
- But if arrays cannot be written/modified, "covariance" is sound!
 - It is only possible to **use** the content of array cells, that is of subtype in place of supertype

Class subtyping

- Subclasses can usually **override some declarations** of the superclass
 - Usually the body of methods
- Assume it is possible to **override both fields and methods**, by changing also their types
 - This will be possible in our FOOL language

Field overriding

- Let's start by overriding fields
 - Class $A\{\dots, T:f, \dots\}$
 - Class B inherits $A\{\dots, T':f, \dots\}$
 - Fields can be seen as array cells
 - So if they can be dynamically modified, their type T cannot be changed in the subclass
 - IN GENERAL FIELD SUBTYPING IS NOT SOUND
(for this reason in Java we cannot override fields)
 - But if they are **immutable** (the fields cannot be modified in FOOL) subtyping is admitted:
 - if $T' \leq T$ (for all fields) then $B \leq A$
- we call these "**covariant fields**"

Method overriding

- Consider now the possibility to override methods by changing the return and, also, the parameter types (not possible in Java):
 - Class $A\{\dots, T:m(T_1:p_1, \dots, T_n:p_n) \{e\}, \dots\}$
 - Class B inherits $A\{\dots, T':m(T'_1:p'_1, \dots, T'_n:p'_n) \{e'\}, \dots\}$
- Consider " $\text{let } x:T \leftarrow y.m(e_1, \dots, e_n) \text{ in } e$ " assuming " y " with static type A and dynamic type B
 - The B return type T' must be usable in place of the A return type T
 - we need $T' \leq T$
 - A parameter types must be usable in place of B parameter types
 - we need $T_i \leq T'_i$

Method overriding (cont.)

- Summarising:
 - if $T_1 \leq T'_1 \dots T_n \leq T'_n$ and $T' \leq T$ (for all methods)
then $B \leq A$
 - The type of m in A is: $T_1, \dots, T_n \rightarrow T$
 - The type of m in B is: $T'_1, \dots, T'_n \rightarrow T'$
 - The **general subtyping rule for functions** is:

$$\frac{T_1 \leq T'_1 \dots T_n \leq T'_n \text{ and } T' \leq T}{(T'_1, \dots, T'_n \rightarrow T') \leq (T_1, \dots, T_n \rightarrow T)}$$

"covariant" output (return) type

"contravariant" input (parameter) types

Comments

- The typing rules use very concise notation
- They are very carefully constructed
- Virtually any change in a rule either:
 - Makes the type system unsound
(bad programs are accepted as well typed)
 - Or, makes the type system less usable
(good programs are rejected)
- But some good programs will be rejected anyway
 - The notion of a good program is undecidable
(we will study this when we will talk about "computability theory")